

PyDPEET: A Python Package for Automated Processing of Battery Measurement Data

Martin Otto^{1*}, Anton Schlösser^{1*}, Daniel Schröder¹, Jan Kalisch¹, Alexander Hinrichsen¹, Cataldo De Simone¹, and Julia Kowal¹

¹ TU Berlin, Institute of Energy and Automation, Electrical Energy Storage Technology (EET), Einsteinufer 11, D-10587 Berlin, Germany   Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Statement of Need

Experimental battery research generates large amounts of measurement data from a wide range of battery cyclers and electrochemical measurement systems. These systems commonly use vendor-specific file formats, column names, units, metadata structures, and representations of experimental procedures. Consequently, substantial preprocessing is often required before measurement data can be analysed. In many research workflows, this preprocessing is implemented through experiment- or device-specific scripts, making analyses difficult to transfer between datasets and measurement systems.

At the same time, battery characterization increasingly relies on combinations of different experiments and analysis methods. Measurements from individual tests often need to be combined into test series or larger measurement campaigns, while subsequent analyses require a consistent representation of quantities such as voltage, current, time, capacity, energy, state of charge, or test steps. Reimplementing these processing steps for individual projects increases development effort and can reduce reproducibility and comparability between studies.

There is therefore a need for a reusable and measurement-system-independent processing framework that transforms heterogeneous raw battery measurement data into a consistent and structured representation for subsequent analysis. Such a framework should reduce experiment-specific preprocessing, facilitate the reuse of analysis methods across datasets, and improve reproducibility and comparability between studies. PyDPEET was developed to address this need

State of the Field

Open-source software for battery data processing and analysis spans a wide range of applications and levels of specialization. Several tools focus on individual characterization or diagnostic methods. For example, dedicated tools exist for electrochemical impedance spectroscopy and distribution of relaxation times analysis ([Huang & Zeier, 2026](#); [Murbach et al., 2020](#); [Wan et al., 2015](#)), degradation mode analysis ([Dubarry et al., 2012](#); [Rehm et al., 2026](#)), and the extraction of model-relevant parameters from techniques such as incremental capacity analysis or galvanostatic intermittent titration ([Randall, 2025](#)).

More general frameworks have been developed to process battery cycling data across different experiments and measurement systems, but they differ in the abstractions used to represent experimental procedures and in the extent of the processing workflow they cover. Cellpy

(Wind et al., 2024) supports multiple battery cyclers, harmonizes their data into a common representation, automatically derives step- and cycle-level information, and provides analysis methods such as ICA and DVA. Recent versions additionally support merging multiple tests, including tests from different cells, into campaign-like data structures. PyProBE (Holland et al., 2025) similarly converts data from several commonly used cyclers into a standardized representation and organizes measurements into cells, procedures, experiments, cycles, and steps. While cycle and event information are represented based on the imported step information, the corresponding experimental information must be manually defined by the user for each test, typically through an accompanying description of the experimental procedure, and is not inferred automatically from the measurement data. BEEP (Herring et al., 2020) follows a different emphasis, combining the structuring of cycling data with feature extraction and workflows for battery lifetime prediction. DATTES (Redondo-Iglesias et al., 2023) provides a broad processing workflow ranging from proprietary cycler-data conversion to automatic segmentation of experiments into operating phases and subsequent analyses including capacity, resistance, impedance, OCV, ICA, and DVA. The battery-data-toolkit focuses primarily on consistently structured battery datasets, metadata, and reusable post-processing functions, providing a common data representation for subsequent analysis. At a more fundamental level, the Battery Data Format defines a standardized and semantically described representation of battery time-series data and provides tools for importing, converting, validating, and cleaning data from different sources.

Consequently, existing frameworks cover substantial parts of the battery data-processing workflow, but use different approaches to describe the experimental structure. Some rely primarily on step and cycle information provided by the measurement system, others require additional descriptions of the experimental procedure, while others automatically segment measurements into operating phases. In particular, the automatic reconstruction of higher-level experimental sequences from measurement data, and their subsequent organization across tests and cells, is not consistently addressed across existing frameworks.

Software	Multi-cycler import	Unified format	Automatic step detection	Automatic higher-level sequence detection	Multi-test series	Multi-cell campaigns	General analysis	Diagnostic analysis
Cellpy	X	X	X	–	X	X	X	X
PyProBE	X	X	(X)	–	X	(X)	X	X
BEEP	X	X	–	–	–	(X)	X	(X)
DATTES	X	X	X	(X)	(X)	(X)	X	X
battery-data-toolkit	(X)	X	–	–	(X)	(X)	X	–
Battery Data Format	X	X	–	–	–	–	(X)	–
PyDPEET	X	X	X	X	X	X	X	X

The comparison distinguishes the frameworks according to their support for data import and harmonization, representation of experimental procedures, automatic reconstruction of operating steps, and aggregation across tests or cells. Most frameworks support the first aspect, whereas the representation and reconstruction of higher-level experimental sequences differ substantially. Cellpy and PyProBE provide structured data models for cycling measurements, while DATTES additionally derives operating phases from the data. In most cases, however, higher-level sequences must be defined by the user or inferred from existing cycler information. Support for combining multiple tests or cells is also available in several frameworks, but is not generally linked to automatic procedure reconstruction.

76 PyDPEET integrates these functions in a common processing workflow. Data from different
77 measurement systems are converted into a unified representation, elementary operating steps are
78 identified from the measurements, and these steps are combined into higher-level experimental
79 sequences. Individual tests can subsequently be grouped into test series and measurements from
80 multiple cells into campaigns while preserving the reconstructed structure. Thus, PyDPEET
81 enables the analysis of heterogeneous datasets even when the original test schedule is unavailable.
82 This enables experimental information to be reconstructed even when the original test schedule
83 or an explicit description of the measurement procedure is unavailable.

84 Software Design

85 PyDPEET's main goal is to provide an easy-to-use, fast, and consistent library that can be
86 easily integrated into existing workflows. Since many scientists in the field of battery research
87 rely on Python scripts to read, analyse, and visualise their data, Python was chosen as the
88 software's primary language. PyDPEET is already available as a package on PyPI and GitHub
89 and provides an autogenerated API layer to give users top-level access to all relevant functions.
90 The automatically updated GitHub Pages provide installation and development guidelines, an
91 API reference generated from extensive docstrings, and in-depth tutorials covering the most
92 common usecases.

93 To keep PyDPEET as lightweight as possible, the list of dependencies is kept at a minimum
94 and additional resource files only consist of PARQUET and TXT files for unit tests and
95 "Sequence Analyzer" precompilation. The codebase itself uses a modular approach to allow
96 future extension within existing submodules (see Fig. 1) as well as the addition of new
97 submodules. The aforementioned top-level function calls ensure that users are unaffected by
98 most internal changes.

99 Fig. 1 shows PyDPEET's general workflow. It uses a straightforward pipeline: first, input
100 battery data is read, converted, and unified ("io" submodule); secondly, time series can be
101 automatically divided into useful chunks for further analysis ("process/sequence" submodule);
102 thirdly, data can be analysed using various functions for typical battery-related quantities
103 ("process/analyze" submodule); lastly, evaluated data can be exported ("io" submodule).
104 The "process/merge" submodule is an optional path for users who want to merge time series
105 from multiple data files into a single file while retaining chronological order. All internal
106 functionality is achieved using the PARQUET file format. This table-based format is fast to
107 process, enables high compression rates for typical input files in the MB-to-GB range, and can
108 be easily converted into other typical formats, e.g., CSV or XLSX.

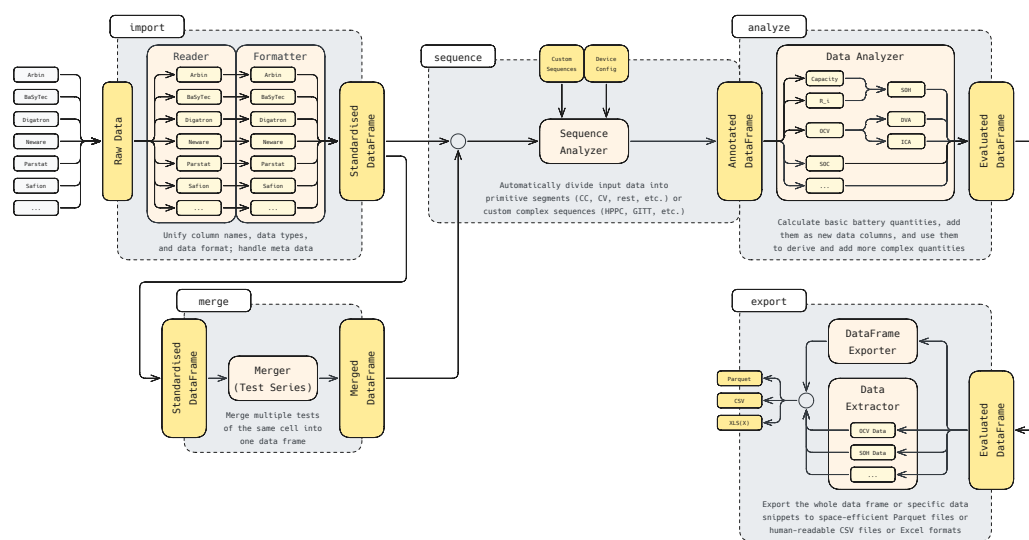


Figure 1: Overview over the Structure and functionalities of the Python Package PyDPEET

In addition to its functionality, a strong focus of the project is maintainability: PyDPEET already contains a test suite which is currently used for basic unit tests. Each commit triggers these tests as well as a linting and formatting stage which uses Ruff and mypy to enforce formal code quality. The merge pipeline adds stages to build and deploy GitHub Pages for the latest state. Releases produce release-specific GitHub Pages and accompanying PyPI version updates.

Research Impact Statement

AI Usage Disclosure

Generative AI tools were used to assist with code suggestions, debugging, documentation, website development, and improving the wording of the manuscript. All AI-assisted output was reviewed, validated, and, where necessary, revised by the authors.

Dubarry, M., Truchot, C., & Liaw, B. Y. (2012). Synthesize battery degradation modes via a diagnostic and prognostic model. *Journal of Power Sources*, 219, 204–216. <https://doi.org/10.1016/j.jpowsour.2012.07.016>

Herring, P., Balaji Gopal, C., Aykol, M., Montoya, J. H., Anapolsky, A., Attia, P. M., Gent, W., Hummelshøj, J. S., Hung, L., Kwon, H.-K., Moore, P., Schweigert, D., Severson, K. A., Suram, S., Yang, Z., Braatz, R. D., & Storey, B. D. (2020). BEEP: A Python library for Battery Evaluation and Early Prediction. *SoftwareX*, 11, 100506. <https://doi.org/10.1016/j.softx.2020.100506>

Holland, T., Cummins, D., & Marinescu, M. (2025). PyProBE: Python Processing for Battery Experiments. *Journal of Open Source Software*, 10(106), 7474. <https://doi.org/10.21105/joss.07474>

Huang, J. D., & Zeier, W. G. (2026). Joint-Domain Impedance Spectroscopy for Solid-State Batteries: Enabling Accelerated Characterization and Data-Driven Insights. *ACS Energy Letters*, 11(2), 986–994. <https://doi.org/10.1021/acsenenergylett.5c03055>

Murbach, M., Gerwe, B., Dawson-Elli, N., & Tsui, L. (2020). Impedance.py: A Python package for electrochemical impedance analysis. *Journal of Open Source Software*, 5(52), 2349. <https://doi.org/10.21105/joss.02349>

- 137 Randall, C. (2025). *Ampworks: Battery analysis tools in Python [SWR-25-39]*. National
138 Renewable Energy Laboratory (NREL), Golden, CO (United States). [https://doi.org/10.](https://doi.org/10.11578/DC.20250313.2)
139 [11578/DC.20250313.2](https://doi.org/10.11578/DC.20250313.2)
- 140 Redondo-Iglesias, E., Hassini, M., Venet, P., & Pelissier, S. (2023). DATTES: Data analysis
141 tools for tests on energy storage. *SoftwareX*, 24, 101584. [https://doi.org/10.1016/j.softx.](https://doi.org/10.1016/j.softx.2023.101584)
142 [2023.101584](https://doi.org/10.1016/j.softx.2023.101584)
- 143 Rehm, M., Natterer, J., Eizenhammer, J., Guenther, M., Kuecher, S., Korkmaz, C., Roehrer,
144 F., & Jossen, A. (2026). How to determine the degradation modes of lithium-ion batteries
145 with silicon–graphite blend electrodes. *Journal of Power Sources*, 670, 239418. [https:](https://doi.org/10.1016/j.jpowsour.2026.239418)
146 [//doi.org/10.1016/j.jpowsour.2026.239418](https://doi.org/10.1016/j.jpowsour.2026.239418)
- 147 Wan, T. H., Saccoccio, M., Chen, C., & Ciucci, F. (2015). Influence of the Discretization
148 Methods on the Distribution of Relaxation Times Deconvolution: Implementing Radial
149 Basis Functions with DRTtools. *Electrochimica Acta*, 184, 483–499. [https://doi.org/10.](https://doi.org/10.1016/j.electacta.2015.09.097)
150 [1016/j.electacta.2015.09.097](https://doi.org/10.1016/j.electacta.2015.09.097)
- 151 Wind, J., Ulvestad, A., Abdelhamid, M., & Mæhlen, J. P. (2024). Cellpy – an open-source
152 library for processing and analysis of battery testing data. *Journal of Open Source Software*,
153 9(97), 6236. <https://doi.org/10.21105/joss.06236>

DRAFT